



Documento Técnico: stack tecnológico y construcción por sprints

Qué tecnologías se usaron para construir el Sistema de Gestión Integral para Conjuntos Residenciales, por qué se eligió cada una y qué se entregó en cada uno de los 15 sprints (S0–S14).

PRODUCTO

Zity · Plataforma web

METODOLOGÍA

Scrum · 16 semanas

RELEASE

v1.0.0-rc

FECHA

Junio 2026

Índice del documento

- 01** **El sistema en una página** — problema, solución, roles y módulos

- 02** **Stack tecnológico y el porqué de cada elección** — frontend, backend, calidad y datos

- 03** **Arquitectura y modelo de datos** — capas, seguridad por rol (RLS) y dominios

- 04** **Cómo se construyó: Scrum y Definition of Done** — equipo, cadencia y calidad escalonada

- 05** **Recorrido por sprints (S0–S14)** — qué se entregó y, sobre todo, qué se usó en cada uno

- 06** **Calidad e ingeniería transversal** — testing, CI/CD, seguridad y privacidad

- 07** **Registro de decisiones técnicas (ADRs)** — las 13 decisiones que dieron forma al sistema

- 08** **Resultados y cierre** — números del proyecto y trabajo futuro

Cómo leer este documento. Es complementario a la presentación institucional *Zity_Presentacion.pptx*. Mientras la presentación cuenta *qué* hace el producto, este documento profundiza en *con qué* se hizo y *por qué*: el razonamiento de ingeniería detrás de cada herramienta y la traza tecnología-por-sprint que sostiene el release candidate.

1 El sistema en una página

Zity nace para resolver la gestión fragmentada de un conjunto residencial: comunicación dispersa en WhatsApp, reclamos sin trazabilidad, cobros manuales y cero visibilidad para la administración. La respuesta es una única plataforma web, en tiempo real, que reúne mantenimiento, facturación, tienda interna y comunicación.

Cuatro roles, seis módulos

Administrador

Centro de control del edificio: gestiona usuarios, solicitudes, facturas, tienda y anuncios.

Residente

Reporta incidencias, paga facturas, compra en la tienda y se comunica desde su panel integral.

Técnico

Atiende las solicitudes que se le asignan y las cierra aportando evidencia fotográfica.

Observador

El dueño del edificio: dashboard ejecutivo de solo lectura que consolida todos los módulos.

Los seis módulos funcionales

Mantenimiento

Facturación

Tienda interna

Comunicación

Usuarios y acceso

Métricas y reportes

Una base segura por diseño. Todo se apoya en Postgres con Row Level Security por rol, notificaciones en tiempo real (Realtime), auditoría inmutable y despliegue continuo. La seguridad no es una capa añadida al final: es una propiedad del modelo de datos desde el Sprint 0.



Pantalla de inicio de sesión — punto de entrada único con autenticación por rol.

2 Stack tecnológico y el porqué

El stack se eligió bajo tres criterios constantes: **un solo proveedor de backend** para no dispersar la seguridad, **tipado de extremo a extremo** para atrapar errores antes de producción, y **presupuesto acotado** (free tier) sin sacrificar prácticas profesionales. Cada elección quedó registrada como ADR.

Frontend

Tecnología	Por qué se usó
React 19	Modelo de componentes declarativo y composable; la UI por rol (admin, residente, técnico, observador) se expresa como árboles de componentes reutilizables. Ecosistema maduro y conocido por el equipo.
Vite 8	Servidor de desarrollo instantáneo (HMR) y <i>build</i> optimizado. Su <i>code-splitting</i> permite el <i>lazy load</i> de módulos pesados (p. ej. las gráficas solo se descargan al abrir métricas), clave para el presupuesto de bundle.
TypeScript 6	Tipado estático de extremo a extremo. Con los <code>enum</code> de Postgres reflejados en tipos, los <code>switch</code> sobre estados/tipos obtienen <i>exhaustive checks</i> en <code>tsc --noEmit</code> , atrapando casos olvidados en compilación, no en producción.
TailwindCSS 4	Estilos por utilidades: diseño consistente y responsive sin hojas de estilo dispersas. La paleta Zity (azul Oxford + warm gold) vive en tokens centralizados.
React Router 7	Enrutado por rol con rutas protegidas: cada usuario aterriza en su propio panel y no puede navegar fuera de su ámbito. Integra <i>lazy routes</i> para dividir el bundle por página.

Backend y datos — Supabase como plataforma única

La decisión estructural del proyecto fue concentrar autenticación, base de datos, archivos, tiempo real y funciones de servidor en **un solo proveedor**, evitando el coste y el riesgo de pegar varios servicios con seguridad desacoplada.

Componente	Por qué se usó
Postgres 17	Base relacional con integridad fuerte: <code>numeric(10,2)</code> para dinero (sin los redondeos de los <i>float</i> de JS), <code>enum</code> para dominios cerrados y <code>UNIQUE</code> que blindan reglas de negocio en la propia BD (anti-duplicados, anti-sobreventa).
Row Level Security	El corazón de la seguridad: cada tabla tiene políticas que garantizan que el residente solo ve lo suyo, el técnico sus asignaciones y el observador agregados de solo lectura. La autorización vive en la BD, no en el cliente.
Auth (JWT)	Autenticación gestionada con verificación por email, recuperación de contraseña y roles. El JWT se valida criptográficamente en el servidor; las sesiones se revocan al bloquear una cuenta.
Realtime	Suscripciones por WebSocket: notificaciones, facturas y anuncios aparecen en vivo sin recargar. Un mismo canal por usuario se reutiliza en tres módulos (S6, S8, S12).

Componente	Por qué se usó
Storage	Fotos y adjuntos en <i>buckets</i> privados con URLs firmadas de caducidad corta. Integrado con la misma Auth y RLS — sin un segundo proveedor de archivos.
Edge Functions (Deno)	Funciones de servidor en TypeScript/Deno para operaciones privilegiadas (invitar, bloquear, notificar por email) que usan <code>service_role</code> y nunca exponen privilegios al navegador.
pg_cron + pg_net	Extensiones de Postgres para automatizar el ciclo de cobro: un <i>job</i> diario marca vencidas y envía recordatorios; uno mensual consolida los pedidos de la tienda en la factura.

¿Por qué Supabase y no un backend a medida? Construir Auth + RLS + Realtime + Storage por separado habría consumido sprints enteros en infraestructura. Supabase entrega todo eso gestionado y dentro del free tier, dejando la capacidad del equipo (≈ 15 h/semana) para construir *producto*. El precio es una dependencia operativa de un proveedor — aceptada y documentada.

Visualización y documentos

Tecnología	Por qué se usó (alternativas descartadas)
Recharts	Gráficas del dashboard como componentes React declarativos, con tipos completos y <i>lazy load</i> para no inflar el bundle inicial. Se descartó SVG a mano (3–4× más trabajo), Chart.js (canvas, menos accesible), ECharts (~ 1 MB, excede el presupuesto) y Visx (demasiado bajo nivel). Ver ADR-006.
pdf-lib	Genera el comprobante de pago PDF 100% en el navegador , con datos que ya pasaron por RLS — sin endpoint ni almacenamiento extra. Se descartó <code>reportlab</code> (Python): habría exigido añadir un runtime ajeno al stack JS/Deno. Ver ADR-011.
react-markdown	Renderiza el cuerpo de los anuncios como markdown seguro : <code>skipHtml</code> , sin <code>rehype-raw</code> y con lista blanca de elementos. Segunda capa de defensa anti-XSS, junto a la sanitización en servidor. Ver ADR-017.

DevOps y calidad

Tecnología	Por qué se usó
GitHub + PRs	Control de versiones y flujo de Pull Request con revisión obligatoria de al menos un developer. Ningún cambio entra a <code>main</code> sin revisión.
GitHub Actions	CI: <i>lint</i> + <i>typecheck</i> + tests como gate de merge . Desde el S10, CD a <i>staging</i> encadenado a la CI (<code>workflow_run</code> + <code>success</code>) para no publicar nunca una build rota. Ver ADR-013.
Vercel	Hosting de la SPA con <i>deploy preview</i> por PR y un dominio estable de <i>staging</i> para la demo y la encuesta de usabilidad.

Tecnología	Por qué se usó
Vitest + Testing Library	Pruebas unitarias y de integración rápidas (comparten el motor de Vite). Cobertura $\geq 60\%$ en core; la lógica pura (KPIs, comprobante, totales) se testea sin tocar la UI.
Playwright	Pruebas end-to-end de flujos críticos (carrito, anuncios) en Chromium. Desde el S11, E2E mínimos en el <i>gate</i> de calidad.
ESLint	Análisis estático de estilo y errores comunes; corre en la CI como primera barrera.

Decisiones de datos transversales. Tres convenciones se aplican en todo el sistema y explican muchas elecciones por sprint: (1) **el dinero es `numeric(10,2)`**, nunca *float*, y la aritmética de montos se hace en SQL; (2) **las operaciones críticas son RPC SECURITY DEFINER** (emisión de facturas, pago, carrito) — atómicas, idempotentes y auditadas en el servidor; (3) **las notificaciones las generan *triggers***, no el cliente, para no debilitar las políticas RLS.

3 Arquitectura y modelo de datos

Zity es una *Single Page Application* de React desplegada en Vercel que habla con Supabase por HTTPS y WebSocket. No hay un servidor intermedio propio: la lógica sensible vive en la base de datos (políticas RLS, triggers, RPC) y en Edge Functions puntuales.

Arquitectura en capas

Frontend · SPA React + Vite (Vercel)

Pages por rol · Components · Custom Hooks · Contexts · Cliente Supabase JS

▲ HTTPS · WebSocket (Realtime) ▼

Supabase · Backend gestionado

Postgres 17 + RLS (tablas, triggers, funciones) · Auth JWT · Realtime · Storage (URLs firmadas) · Edge Functions (Deno)

▲ gate de calidad ▼

CI / CD · GitHub Actions

lint + typecheck + tests como gate → deploy automático a Vercel → verificación de `/health`

Modelo de datos por dominio

Dominio	Tablas principales	Garantías en la BD
Usuarios y acceso	<code>usuarios</code> , <code>invitaciones</code> , <code>edificios</code> , <code>unidades</code>	Perfil 1-1 con <code>auth.users</code> ; invitación con token y caducidad 48 h.
Mantenimiento	<code>solicitudes</code> , <code>asignaciones</code> , <code>historial_estados</code>	Código legible <code>ZIT-NNN</code> por trigger; cada cambio de estado deja rastro.
Facturación	<code>facturas</code> , <code>metodos_pago</code> , <code>facturas_secuencia</code>	<code>numeric(10,2)</code> ; <code>UNIQUE(residente, tipo, periodo)</code> ; número <code>F-AAAA-MM-NNN</code> .
Tienda	<code>productos</code> , <code>pedidos</code> , <code>pedido_items</code>	Baja lógica (<code>activo</code>); FK <code>ON DELETE RESTRICT</code> ; descuento de stock atómico.
Comunicación	<code>anuncios</code> , <code>anuncio_lecturas</code> , <code>notificaciones</code>	Estado de lectura con PK compuesta; cuerpo sanitizado en servidor antes de persistir.
Transversal	<code>audit_log</code>	Inmutable (<i>write-only</i>): solo lo escribe el backend con <code>service_role</code> ; solo el admin lo lee.

Seguridad por diseño

RLS por tabla y rol

La autorización se evalúa en Postgres. Aunque alguien llamara a la API REST directamente, solo recibiría las filas que su rol permite.

service_role solo en el servidor

Las operaciones privilegiadas (invitar, bloquear, auditar) viven en Edge Functions/triggers; la clave de servicio nunca llega al navegador.

Auditoría inmutable

`audit_log` registra quién hizo qué, cuándo y sobre qué entidad. Es *write-only* desde el backend y de lectura exclusiva del admin.

Storage con URLs firmadas

Las fotos viven en buckets privados; se sirven con URLs firmadas que caducan en una hora. Nada es indexable públicamente.

4 Cómo se construyó: Scrum y DoD

Quince sprints (S0–S14) en 16 semanas, con un enfoque funcional: cada sprint entrega un módulo demostrable y la parte técnica se reparte como *chore* obligatorio de 1–2 h. La Definition of Done sube en tres escalones y la calidad nunca baja.

Equipo

PO: Jaqueline Álvarez Rocca · **SM:** Carlos Meza Peláez

Developers: Leonardo Cortez · Yoel Gonza · Steven Santiago

Cadencia

Planning, Review y Retrospective por sprint; Refinement de 30 min desde el S5. Cada sprint cierra con una demo demostrable. Capacidad ≈15 h/sem, velocidad observada ≈13,2 h.

La Definition of Done sube en tres escalones

Nivel	Sprints	Criterios que añade (hereda los anteriores)
DoD v1 Baseline	S0–S3	CI en verde (lint + unit tests) · PR revisado por un developer · README y datos demo (<code>seed</code>) · <i>deploy preview</i> en Vercel por PR.
DoD v2 Calidad + integración	S4–S10	Tests de integración en flujos críticos · cobertura ≥60% en core · endpoint <code>/health</code> en staging · Playwright + 1 E2E como gate · <code>tsc --noEmit</code> sin errores.
DoD v3 Release candidate	S11–S14	E2E mínimos en CI · no-PII en logs (solo IDs) · checklist OWASP Top 10 · Lighthouse ≥80 en rutas clave · release candidate etiquetado.

El *chore* técnico es obligatorio, no opcional. Cada sprint funcional reserva 1–2 h para pagar deuda técnica o subir el listón de calidad (instalar Playwright, vista materializada, no-PII, OWASP, Lighthouse). Así la DoD avanza sin frenar la entrega de producto.

5 Recorrido por sprints (S0–S14)

El núcleo de este documento. Por cada sprint: el objetivo, lo que se entregó y —lo más importante— **qué tecnologías y técnicas concretas se usaron** para lograrlo. Observa cómo cada pieza se reutiliza en sprints posteriores en lugar de reinventarse.

S0 Setup técnico y cimientos

DoD v1 Semana 2

Dejar el entorno reproducible y la fábrica de software lista antes de escribir una sola feature.

- Repositorio y flujo de PR con revisión obligatoria de al menos un developer.
- Supabase aprovisionado (Postgres 17, Auth, Storage, Realtime) y ADRs base.
- Datos demo reproducibles y CI en verde como condición de merge.

QUÉ SE USÓ

Git + GitHub

GitHub Actions (CI)

ESLint

Vitest

Supabase (provisión)

Vercel (preview por PR)

Vite

npm run seed

S1 Módulo de Autenticación

DoD v1 Semana 3

Seguridad desde el día uno: cada rol entra y aterriza en su propio panel.

- Registro en 2 pasos y login por rol (admin, residente, técnico).
- Recuperación de contraseña por email con enlace válido 60 minutos.
- Rutas protegidas y estados de cuenta: pendiente → activo → bloqueado.

QUÉ SE USÓ

Supabase Auth (JWT)

Verificación por email

React Router (rutas protegidas)

VITE_SITE_URL (links de correo)

React Context (sesión)

S2 Panel admin y gestión de usuarios

DoD v1 Semana 4

El centro de control del edificio y las operaciones privilegiadas, ejecutadas de forma segura en el servidor.

- Gestión de usuarios: ver, filtrar, invitar (token 48 h) y aprobar.
- Bloqueo/desbloqueo con cierre inmediato de la sesión (revoke).

QUÉ SE USÓ

Edge Functions (Deno)

service_role

RPC revoke_user_sessions

ban_duration

audit_log

React (AdminShell, ProtectedRoute)

S3 Mantenimiento v1 — crear y ver

DoD v1 Semana 5

El núcleo del producto: el canal único de reportes que reemplaza WhatsApp y las notas en papel.

- El residente crea solicitudes con foto (cámara/galería), tipo, categoría y prioridad.
- Código legible `ZIT-NNN` autogenerado; vista admin en tabla filtrable.

QUÉ SE USÓ

Supabase Storage (bucket privado)

URLs firmadas (1 h)

Trigger Postgres + secuencia (ZIT-NNN)

Triggers de auditoría/historial

API de cámara (móvil)

S4 Mantenimiento v2 — ciclo completo

DoD v2 Semana 6

Cerrar el ciclo de una solicitud y devolver el control al residente (mejora pedida por el profesor en la Review del S3).

- Asignación de técnico y vista del técnico con sus tareas; flujo pendiente → en progreso → resuelta → cerrada.
- Confirmación del residente: confirma o rechaza con nota; **3 rechazos escalan al admin**.

QUÉ SE USÓ

Máquina de estados en BD

RLS UPDATE multi-rol

Índice parcial (pendientes de confirmación)

Trigger de defensa en profundidad

Helper cambiarEstadoSolicitud

Vitest (integración, cobertura ≥60%)

S5 Trazabilidad y auditoría

DoD v2 Semana 7

Que cada acción crítica quede registrada de forma inmutable para poder rendir cuentas.

- Log de auditoría visible (quién, qué, cuándo, sobre qué entidad) e historial completo de cada solicitud.
- Perfil editable (nombre, teléfono, contraseña).

QUÉ SE USÓ

audit_log write-only (service_role)

RLS de lectura solo-admin

historial_estados (línea de tiempo)

Triggers SECURITY DEFINER

S6 Comunicación en tiempo real

DoD v2 Semana 8

Que la interfaz se actualice sola ante cambios de estado y asignaciones. La infraestructura creada aquí se reutiliza en facturas (S8) y anuncios (S12).

- Notificaciones en vivo con bandeja de campana e indicador de no leídos recalculado desde la BD.
- Email de aviso (Edge Function; Resend en producción, dry-run sin API key).

QUÉ SE USÓ

Supabase Realtime (WebSocket)

Canal por usuario + NotificacionesContext

Reconexión exponencial

Trigger SECURITY DEFINER (genera notificaciones)

Edge Function + Resend

S7 ★ Métricas y Dashboard visual

DoD v2 Semana 9

Dar a la administración los datos para decidir: KPIs, tiempos de resolución y tendencias.

- Panel de KPIs (total, pendientes, en proceso, resueltas) y tiempos promedio, mediana (P50) y P95.
- Tres gráficas (por tipo, top categorías, tendencia mensual) y exportación a CSV.

QUÉ SE USÓ

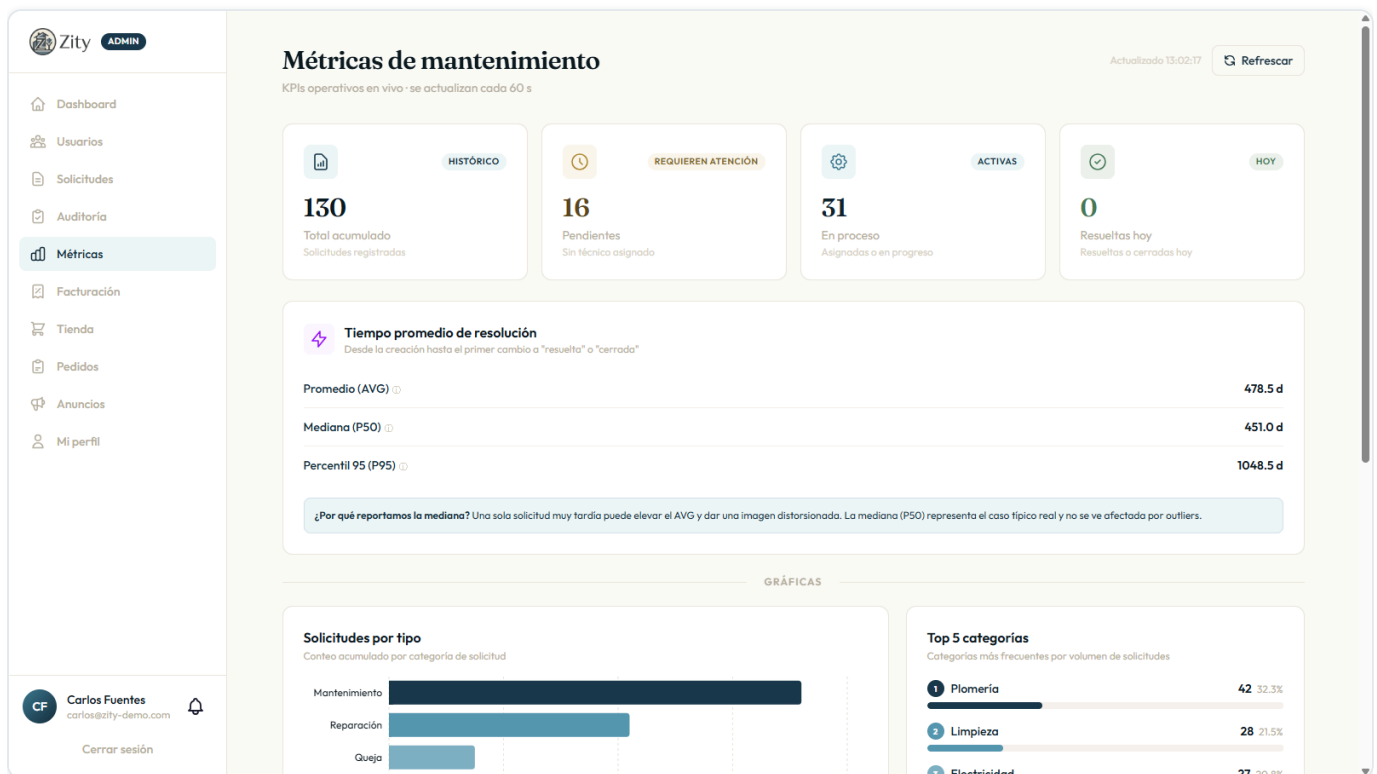
Recharts (lazy load anidado)

Lógica de KPIs pura (testable)

Vista materializada Postgres (600 ms → 20 ms)

Exportación CSV

Playwright (instalación + 1.er E2E)



S7 · Dashboard de métricas con gráficas Recharts y la paleta Zity, servidas desde una vista materializada.

S8 Facturación v1 — emisión y vista

DoD v2 Semana 10

Abrir el dominio de facturación: emitir cargos por tipo, individual o en lote, sin duplicados ni errores de redondeo.

- Emisión por tipo (luz, agua, pensión, multas), individual o en lote a todos los residentes activos en una **transacción atómica**.
- Vista del residente con desglose y número legible `F-AAAA-MM-NNN` ; aviso Realtime al emitir.

QUÉ SE USÓ

Enums Postgres + numeric(10,2)

UNIQUE (anti-duplicados)

Trigger de numeración + secuencia

RPC emitir_facturas_lote (atómica)

Trigger → notificación Realtime (reusa S6)

RLS con get_userRol()

S9 Facturación v2 — cobros y recordatorios

DoD v2 Semana 11

Cerrar el ciclo de cobro con automatización y un comprobante descargable; sembrar la observabilidad.

- Marcar como pagada (operación idempotente) y totales del periodo (emitido/cobrado/pendiente/vencido).
- Job diario que marca vencidas y recuerda 3 días antes; comprobante PDF descargable y endpoint `/health` .

QUÉ SE USÓ

RPC idempotente (FOR UPDATE)

pg_cron (job diario)

Zona horaria America/Lima

pg_net + Resend (fire-and-forget)

pdf-lib (comprobante en cliente)

Endpoint /health (DB+Auth+Storage)

S10 Tienda interna v1 — catálogo

DoD v2 Semana 12

Abrir la tienda con un catálogo gestionable y automatizar el despliegue (cierra la DoD v2).

- Gestión de catálogo (alta/baja/edición con foto, precio y stock) con baja lógica reactivable; catálogo del residente con filtros y búsqueda.
- CD a staging: deploy automático en cada merge a `main` con la CI como gate.

QUÉ SE USÓ

Enum producto_categoria + numeric(10,2)

Baja lógica (activo)

FK ON DELETE RESTRICT

Bucket productos-fotos (2 MB)

Búsqueda con debounce

GitHub Actions (workflow_run) + Vercel CLI

Verificación /health post-deploy

S11 Tienda interna v2 — carrito + factura

DoD v3 Semana 13

Cerrar el ciclo de la tienda: carrito robusto, sin sobreventa, integrado con la facturación mensual. Inicia la DoD v3.

- Carrito con descuento de stock **atómico** (imposible sobrevender, verificado con prueba de concurrencia) y precio fijado en servidor.
- El pedido se consolida como línea *tienda* en la factura mensual del residente.

QUÉ SE USÓ

Carrito como pedido borrador en BD RPC SECURITY DEFINER (carrito + confirmar)
SELECT ... FOR UPDATE (anti-sobreventa) ALTER TYPE (factura_tipo 'tienda') pg_cron mensual (cierre)
Chore no-PII (logs solo IDs)

S12 Tablón de anuncios del edificio

DoD v3 Semana 14

Dar al edificio un canal de comunicación oficial, en vivo y a prueba de XSS.

- El admin publica comunicados (categoría, prioridad, imagen/PDF, fijar, vigencia); el residente los recibe en vivo con badge "Nuevo" y los marca como leídos.
- Notificación selectiva (solo prioridad importante/urgente) para evitar el spam.

QUÉ SE USÓ

anuncio_lecturas (PK compuesta) Sanitización en servidor (función + trigger)
react-markdown seguro (skipHtml) Realtime (reusa S6) Bucket anuncios-adjuntos
Chore OWASP A01/A03/A05

S13 Panel integral del residente

DoD v3 Semana 15

Unificar la experiencia del residente en un home con todo lo importante a un clic, sin inventar endpoints.

- Home con 3 tarjetas (solicitudes activas, facturas pendientes, pedidos del mes) que navegan al detalle; diseño responsive.
- Métodos de pago (mis tarjetas + pago de factura con tarjeta) y sesiones activas con cierre de todas al cambiar la contraseña.

QUÉ SE USÓ

Vistas Postgres ligeras (totales) Reuso de datos existentes CRUD de métodos de pago
Sesiones activas (session_id) Chore Lighthouse ≥80

S14 ★ Dashboard ejecutivo + Release Candidate

DoD v3 Semana 16

El broche: una vista de solo lectura para el dueño que consolida todo el producto, y el etiquetado del release candidate.

- Rol "observador" (solo lectura) y dashboard ejecutivo que integra Mantenimiento + Finanzas + Tienda en una sola vista.
- Consolida KPIs (S7) + facturas (S8/S9) + pedidos (S10/S11) sin inventar datos; `v1.0.0-rc` + smoke tests post-deploy.

QUÉ SE USÓ

Nuevo rol observador (RLS solo lectura)

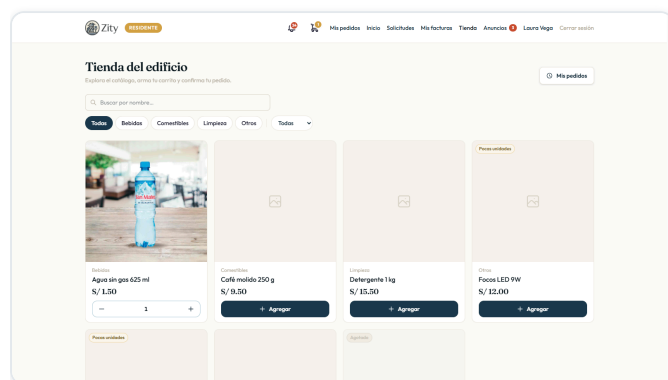
Recharts (reuso S7)

Consolidación de módulos existentes

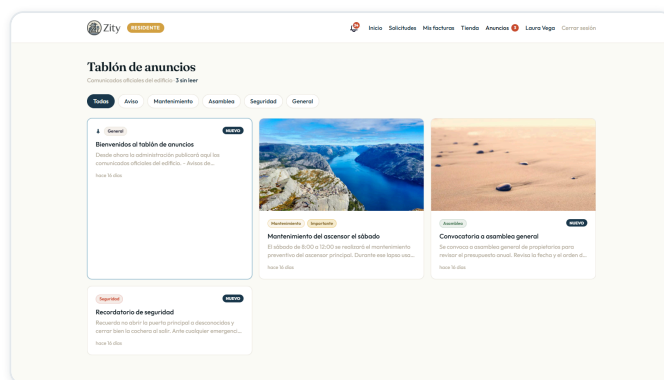
Smoke tests post-deploy

Tag v1.0.0-rc

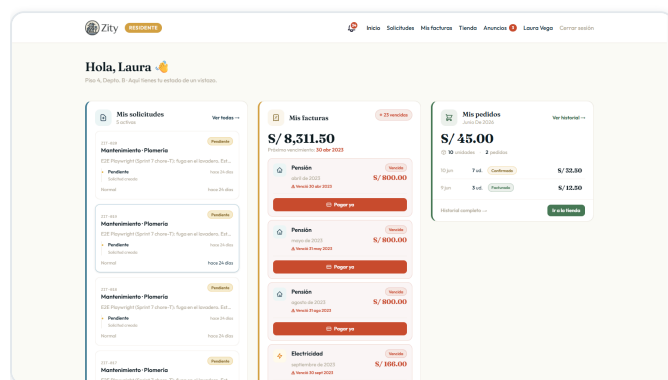
Evidencia visual del producto construido



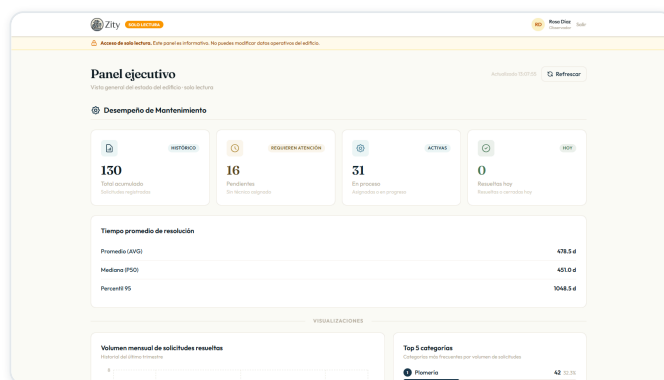
S10-S11 · Tienda del residente: catálogo, carrito y descuento de stock atómico.



S12 · Tablón de anuncios en vivo, con cuerpo markdown sanitizado.



S13 · Panel integral del residente con las tres tarjetas resumen.



S14 · Dashboard ejecutivo (rol observador) que consolida todos los módulos.

6 Calidad e ingeniería transversal

Más allá de las features, estas prácticas atraviesan todos los sprints y son las que convierten un prototipo en un *release candidate* defendible.

Testing

Vitest para unit + integración con cobertura $\geq 60\%$ en módulos core. Playwright para E2E de flujos críticos. Pruebas de concurrencia específicas (stock: no sobreventa).

Vitest

Testing Library

Playwright

CI / CD

GitHub Actions con lint + typecheck + tests como gate. Deploy automático a Vercel en cada merge y verificación `/health` (DB + Auth + Storage) post-deploy.

GitHub Actions

Vercel

`/health`

Seguridad

Row Level Security por tabla y rol; `service_role` confinado a Edge Functions; checklist OWASP Top 10 (A01 acceso, A03 XSS con sanitización en servidor, A05 configuración).

RLS

OWASP Top 10

Sanitización en servidor

Privacidad y rendimiento

No-PII en logs (solo IDs). Lighthouse ≥ 80 en rutas principales. Auditoría inmutable y todas las decisiones documentadas como ADR.

No-PII

Lighthouse ≥ 80

ADRs

La calidad como propiedad acumulada. El endpoint `/health` (S9) habilitó la verificación post-deploy automática (S10); el patrón Realtime (S6) sirvió a facturas y anuncios; la sanitización (S12) quedó como función reutilizable. Ninguna pieza de calidad fue desechable: cada una se convirtió en cimiento de la siguiente.

7 Registro de decisiones técnicas

Cada elección no trivial se registró como ADR (Architecture Decision Record): contexto, opciones evaluadas y justificación. Esta es la traza que respalda el "por qué" de las secciones anteriores.

ADR	Decisión	Justificación esencial
005	Imágenes con Supabase Storage	Integrado con Auth y RLS, URLs firmadas, sin coste extra. Frente a Cloudinary (de pago, auth desacoplada) y Base64 en BD (infla y degrada).
006	Gráficas con Recharts	API React declarativa, tipada, con lazy load. Frente a SVG a mano (3–4× más trabajo), Chart.js, ECharts (~1 MB) y Visx (muy bajo nivel).
007	Modelo de facturas	Enums + <code>numeric(10,2)</code> (sin redondeos), <code>UNIQUE</code> anti-duplicados y numeración por trigger atómico.
008	Confirmación del residente	El residente —no el técnico— cierra la solicitud; 3 rechazos escalan al admin. Trazabilidad real, sin cierres unilaterales.
009	Notificaciones Realtime	Un canal por usuario + generación por trigger <code>SECURITY DEFINER</code> : seguro (nadie fabrica notificaciones ajenas), menos tráfico, atómico.
010	Ciclo de cobro (jobs)	Pago como RPC idempotente; un job <code>pg_cron</code> diario para vencidas y recordatorios; fechas en <code>America/Lima</code> .
011	Comprobante PDF con pdf-lib	Generado en el cliente con datos ya filtrados por RLS, sin backend extra. Frente a <code>reportlab</code> (exigiría runtime Python).
012	Modelo de la Tienda	Baja lógica + FK <code>ON DELETE RESTRICT</code> para blindar el historial de pedidos; enums y <code>numeric(10,2)</code> .
013	CD a staging	Deploy encadenado a la CI (<code>workflow_run</code> + <code>success</code>) para no publicar builds rotas; secrets fuera del repo; verificación <code>/health</code> .
014	Carrito con descuento atómico	Carrito como borrador en BD; <code>FOR UPDATE</code> + <code>UPDATE ... WHERE stock ≥ cantidad</code> : imposible sobrevender.
015	Integración pedido → factura	Nuevo valor de enum <code>tienda</code> reusa toda la infra de facturas; idempotente por el <code>UNIQUE</code> ; trazable vía <code>factura_id</code> .
016	Modelo del Tablón de anuncios	Estado de lectura con PK compuesta (contador siempre derivado de la BD); baja lógica; RLS que filtra vigencia en la propia BD.
017	Sanitización y aviso Realtime	Defensa en profundidad anti-XSS: sanitización en servidor (trigger) + render seguro en cliente; notificación solo de lo importante.

8 Resultados y cierre

De la comunicación dispersa a un sistema único y trazable. Zity integra mantenimiento, finanzas y tienda con tiempo real, seguridad por rol y un dashboard ejecutivo que consolida todo el producto.

15

Sprints (S0–S14)
entregados y
demostrados

6

Módulos funcionales, de
mantenimiento a
ejecutivo

4

Roles con RLS: admin,
residente, técnico,
observador

v1.0.0-rc

Release candidate con
CD a staging

Calidad verificada

Cobertura $\geq 60\%$ core

E2E en CI (Playwright)

Lighthouse ≥ 80

OWASP Top 10

No-PII en logs

/health en staging

Auditoría inmutable

Trabajo futuro (post-curso)

Twilio Verify (2FA)

Segundo factor de autenticación para reforzar el acceso.

Web Push + presencia

Avisos nativos y estado en línea de los usuarios.

Resumen diario por email

Digest automático para el residente.

Cobertura E2E publicada

Reporte de calidad visible como evidencia.

El criterio que guió todas las decisiones. Concentrar la plataforma en Supabase, poner la seguridad y la lógica crítica en la base de datos, tipar de extremo a extremo y subir la calidad por escalones (DoD v1→v2→v3) permitió a un equipo pequeño, con ≈ 15 h semanales, entregar un producto integrado y defendible en 16 semanas. Cada tecnología se eligió por encajar en ese criterio — y se documentó por si hubiera que revisarla.